Earlier you learned that establishing a database connection has some costs, and adds some latency to the query. Some of the costs are:

- Negotiating SSL
- Authentication

Without reuse of a connection from a pool, these costs are paid for every new query. With a connection pooler, these costs are not paid for each new query or even each new connection. PgBouncer holds connections open to PostgreSQL so that clients like Active Record can reuse them.

On Mac OS, install PgBouncer using Homebrew.[10] Run this command from your Terminal:

sh/pgbouncer_install.sh

```
brew install pgbouncer
```

After PgBouncer is installed, run the following commands to inspect the pgbouncer process and restart it.

sh/pgbouncer_commands.sh

```
brew services info pgbouncer
brew services restart pgbouncer
```

PgBouncer runs on port 6432 which is exactly 1000 higher than the default PostgreSQL port of 5432. Exciting trivia for your next PostgreSQL cocktail party!

Next, set up a PostgreSQL user that will act as the application user to connect with. Run these statements from psql:

sql/create_test_user.sql

```
DROP ROLE IF EXISTS app_user;
CREATE USER app_user WITH PASSWORD 'insecurepwd';
```

Configure the application user. Add the admin_users = app_user section below to make app_user an admin user. Edit the .ini configuration file by opening it in your text editor. Find the file path by running brew info pgbouncer. After changing the config file, run pgbouncer -R /usr/local/etc/pgbouncer.ini from your Terminal to restart PgBouncer.

sh/pgbouncer_ini_config.sh

```
[pgbouncer]
 listen_port = 6432
 listen_addr = localhost
 auth_type = md5
 auth_file = /usr/local/etc/userlist.txt
```

10. https://brew.sh